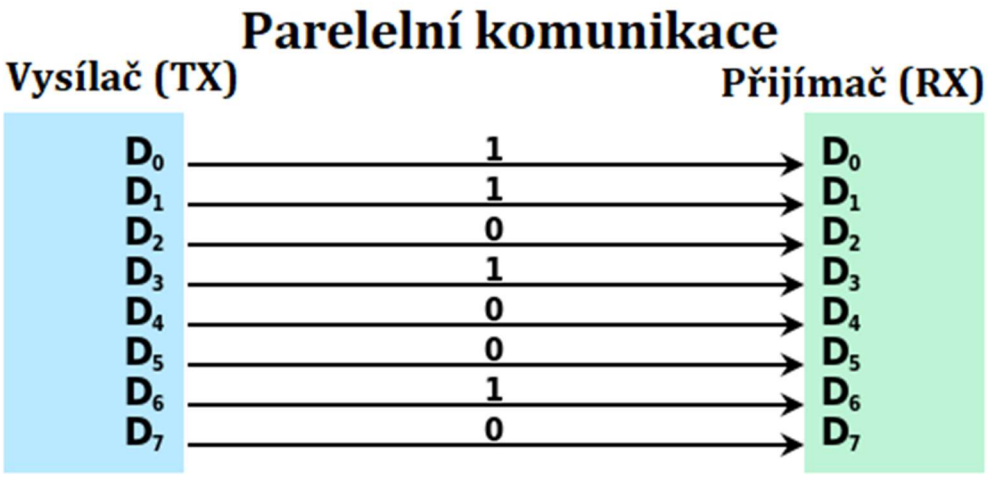
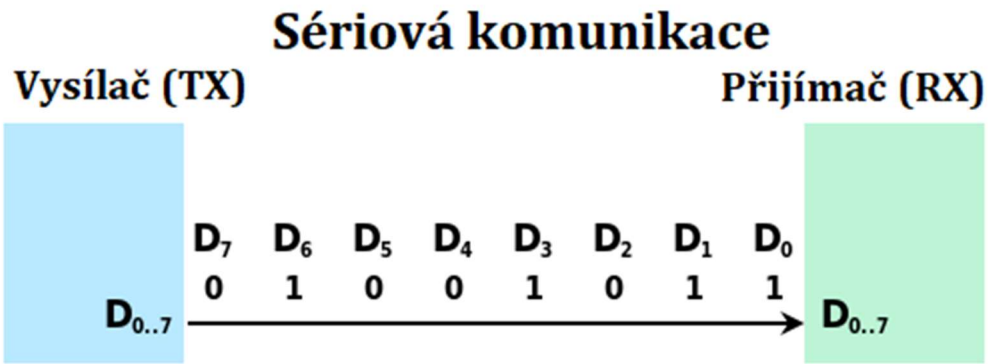
Sériová komunikace I²C, SPI, UART

Sériová komunikace je způsob přenosu dat, při kterém jsou jednotlivé bity postupně (sekvenčně) posílány po komunikačním kanálu nebo sběrnici.

Opakem sériové komunikace je paralelní komunikace, kde je několik bitů přenášeno najednou pomocí několika paralelních přenosových kanálů.





I^2C

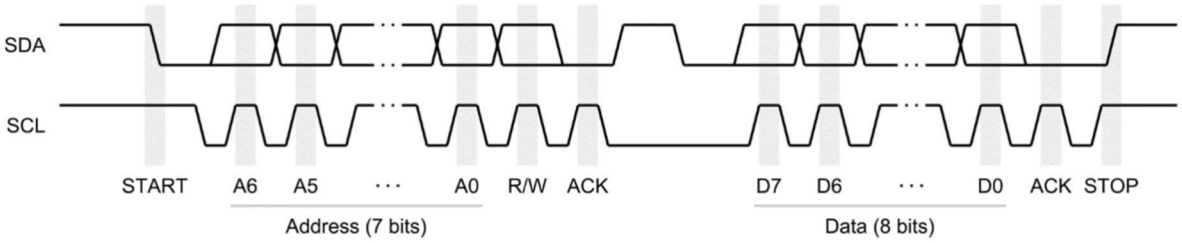
IIC (zkratka Inter-Integrated Circuit) je sériový osmibitový komunikační standard často používaný v embedded systémech (amatérských – Arduino, i průmyslových – PLC (třeba SIEMENS SIMATIC IOT2000)) (Embedded systém – jednoúčelový systém se zabudovaným počítačem (Arduino s display modulem ovládající motor, Siemens LOGO! …), eg. PC z něj nelze odpojit, tvoří jedno zařízení).

Komunikace probíhá pomocí 2 vodičů – SCK/SCL (Serial clock line) a SDA (Serial data). Mimo dobu komunikace se linka nechává volná, avšak pomocí pull-up rezistorů v logické „1“ (tj. u Arduina 5V, u ESP 3,3V, …).

Zařízení jsou rozdělena na 1 master a až 127 slave. Master zařízením může být kterékoliv, avšak vždy jen jedno v jeden čas.

Zpráva se skládá z:
Start bitu - SDA je přepnuto na log. 0, SCK po něm na log. 0 -> Upozornění zaříz. na komunikaci.
Adresy (7 bit) - Master odesílá adresu slave, se kterým navazuje komunikaci (2_7 = 128 (-1) -> 127 zaříz.)
Čtení/zápis b. - 8. bit adresy je R/W bit signalizující čtení / zápis dat do slave.
Potvrzovac. b. - Pokud je na lince slave s příslušnou adresou, potvrzuje signálem ACK (SDA -> 0)
1. Zpráva (8 b.) - 1. Byte zprávy pro / ze slave zařízení
Potvrzovac. b. - Potvrzení přijetí zprávy (při zápisu potvrzuje slave, při čtení master)
n. Zpráva (8 b.) - … N. Byte zprávy ze / do slave zařízení
Stop bitu - SCK je přepnuto na log. 1 a ponecháno, po něm je SDA přepnuto také do log. 1

Časový diagram:



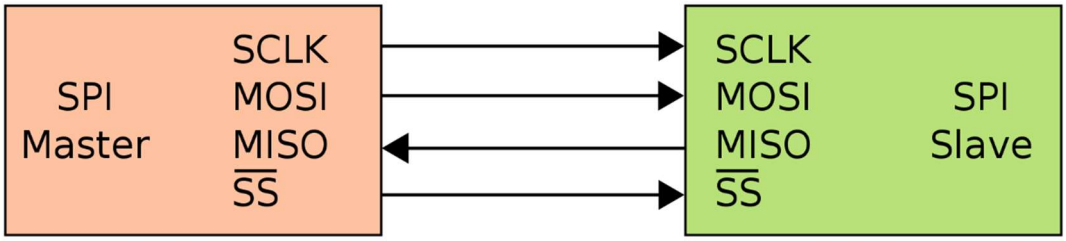
Průběh komunikace z pohledu master zařízení:

1. Zahájení komunikace
2. Odeslání adresy zařízení + bit čtení/zápis
3. Čekání na ACK (Master uvolní SDA na log. 1 a slave uzemněním na log. 0 potvrdí přijetí zprávy, pak ji zase uvolní)
4. Čtení/zápis:
a. Čtení: Slave mi posílá 8 bitů dat, pak uvolní SDA a já potvrdím signálem ACK
b. Zápis: Posílám do slave 8 bitů dat, on potvrdí příjem signálem ACK
5. Čtení/zápis dalších byte
6. Ukončení komunikace

Jelikož je R/W signál posílán již na začátku, je při komunikaci možnou pouze číst / pouze zapisovat. Pro zápis dat po čtení / čtení po zápisu je potřeba komunikaci ukončit a zahájit znovu.

SPI

Rozhraní SPI (Serial Peripheral Interface) je osmibitové sériové periferní rozhraní, které umožňuje komunikaci mezi mikrokontroléry (Master – RPi Pico, Arduino, ESP32…) a ostatními integrovanými obvody (Slave - A/D převodníky, displeje, různé senzory…). Typická konfigurace SPI obsahuje jeden master (hlavní zařízení) a jedno nebo více slave zařízení, přičemž master řídí komunikaci.



1. SCLK (Serial Clock): Určuje synchronizační hodinový signál (takt) pro přenos dat.
2. MOSI (Master Output Slave Input): Slouží k přenosu dat z mastera do slave zařízení.
3. MISO (Master Input Slave Output): Přenáší data ze slave zařízení zpět do mastera.
4. SS (Slave Select) nebo CS (Chip Select): Signál použitý k označení, které slave zařízení má být aktivní pro komunikaci.

Komunikace je plně obousměrná mezi masterem a slave zařízeními a probíhá v rámci jednoho cyklu taktu. Data jsou posílána po jednotlivých bitech, kde master generuje hodinový signál, zatímco posílá data do slave zařízení pomocí MOSI a získává data zpět pomocí MISO.

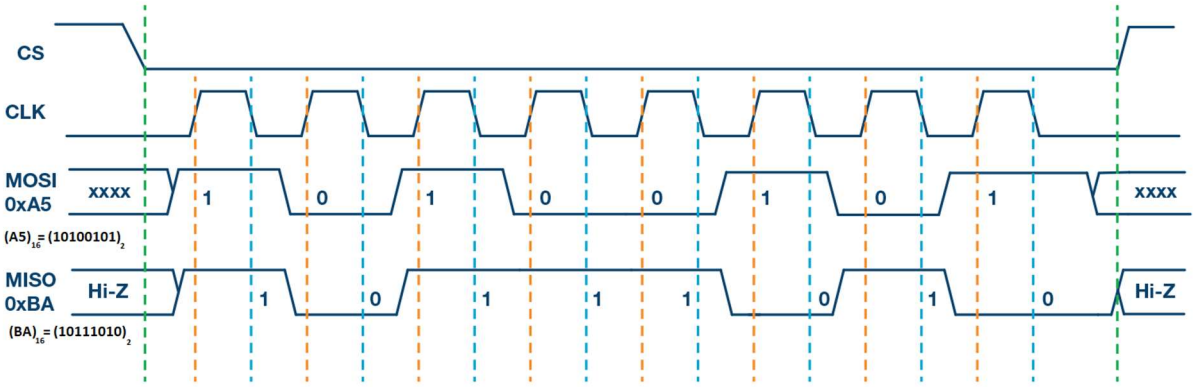
Výhody SPI oproti jiným komunikačním rozhraním (I2C, UART):

1. Vyšší rychlost přenosu dat: SPI obvykle nabízí vyšší přenosovou rychlost než jiná rozhraní jako I2C a UART, protože neexistuje omezení adres nebo složitých protokolů.
2. Jednoduchost a efektivita: SPI je jednodušší a efektivnější v přenosu dat v porovnání s I2C nebo UART, což umožňuje rychlejší komunikaci bez složitých protokolů adresování.
3. Plně duplexní komunikace: V SPI může master i slave zařízení přenášet data současně, což zvyšuje efektivitu komunikace.

Průběh komunikace:

1. Pro komunikaci Master nastaví log. 0 na CS zařízení, se kterým chce komunikovat.
2. Pak začne generovat hodinový signál na SCLK a v té chvíli vyšlou obě zařízení svoje data (přes MOSI a MISO).
3. Jakmile jsou data vyslána, může komunikace dále pokračovat:
   a. Master dále dodává hodinový signál, hodnota SS se nemění
   b. nebo může být ukončena: Master přestane vysílat hodinový signál a nastaví SS do log. 1.

4. Délka vyslaných dat je buď 8bit (Byte) a nebo 16bit

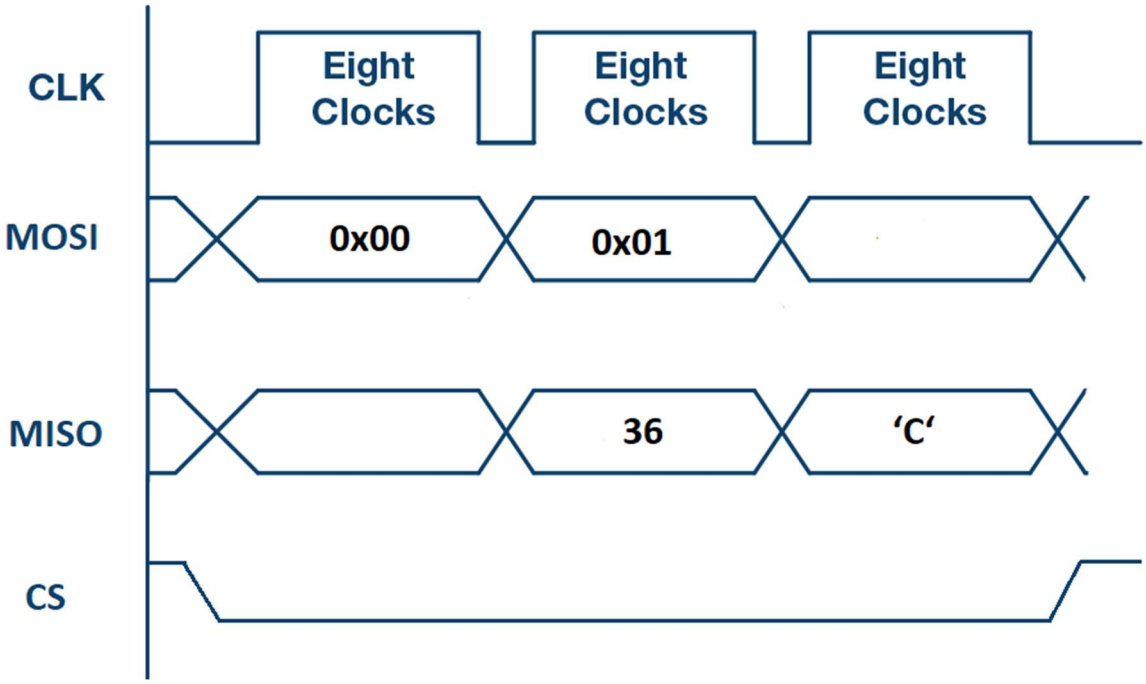


Adresa	Hodnota	Operace
0x00	36	Read
0x01	‘C‘	R
0x02	44	R
0x03	‘%‘	R

Tabulka registrů slave zařízení (teplota a vlhkost)

MASTER pošle po MOSI signál 0x00 v době jednoho taktu.

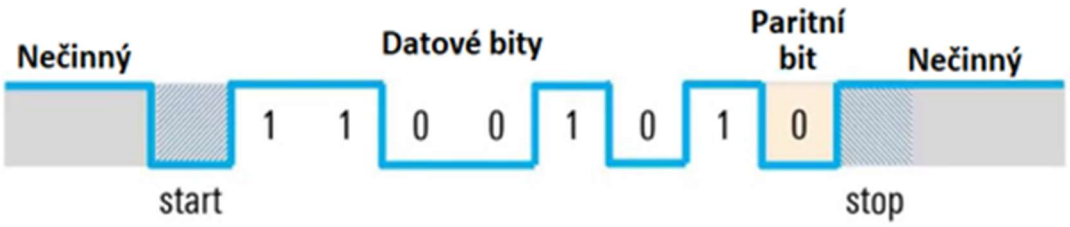
SLAVE pošle po MISO signál 36 v době jednoho taktu.



UART

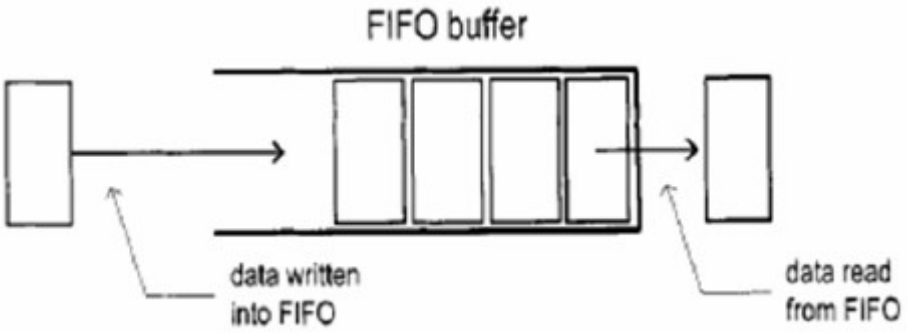
= hardwarový komunikační protokol používaný pro přenos dat mezi dvěma zařízeními bez společného hodinového signálu. Je to jeden z nejstarších a nejjednodušších komunikačních protokolů, který se používá v počítačích a dalších digitálních systémech. UART převádí paralelní data na sériová a přenáší je jedním bitem po druhém. Pro správnou komunikaci je nezbytná shoda v přenosové rychlosti a struktuře paketů mezi odesílajícím a přijímajícím UARTem.

- Přenáší data v tzv. **Rámci dat**
  - Rámec dat je základní jednotka dat v UART komunikaci. Začíná startovacím bitem, následují data (obvykle 5-9 bitů), poté může následovat volitelný paritní bit pro detekci chyb, a nakonec stopovací bit. Paritní bit je použit k zajištění, že počet jedniček v datech je sudý (při sudé paritě) nebo lichý (při liché paritě). Pokud příjemce spočítá jiný počet jedniček, ví, že došlo k chybě při přenosu.



- Využívá **Asynchronní komunikaci**
  - V asynchronní komunikaci neexistuje žádný společný hodinový signál mezi odesílatelem a příjemcem, který by určoval, kdy mají být data odeslána. Místo toho odesílatel označí začátek každého rámce dat startovacím bitem a konec stopovacím bitem. Příjemce pak sleduje tyto bity, aby věděl, kdy začít a kdy přestat číst data.
- **Rychlost přenosu**
  - Rychlost přenosu, nebo baud rate, je počet bitů, které jsou přenášeny za sekundu. Je důležité, aby **odesílatel a příjemce byli nastaveni na stejnou rychlost přenosu**, jinak by mohlo dojít k chybám při přenosu dat.
- UART může být implementován jako fyzický obvod (například jako integrovaný obvod) nebo jako virtuální zařízení v softwaru. Fyzický UART obvykle obsahuje registr pro příjem a odesílání dat a další registry pro nastavení různých parametrů, jako je rychlost přenosu.

- Využívá **FIFO Buffery**:
  - UARTy často používají FIFO (First In, First Out) buffery pro ukládání přijatých a odesílaných dat. To umožňuje UARTu přijímat nebo odesílat více dat najednou, což zlepšuje efektivitu komunikace.



- Může využívat **Handshaking**
  - Některé UARTy používají signály handshakingu, jako je RTS (Request to Send) a CTS (Clear to Send), aby koordinovaly přenos dat. To může pomoci předcházet přetečení bufferu, když odesílatel posílá data rychleji, než je příjemce schopen je zpracovat.

    1.RTS (Request to Send): Tento signál je generován vysílacím zařízením a slouží k označení, že je připraveno na přenos dat.

    2. CTS (Clear to Send): Tento signál je generován přijímacím zařízením a indikuje, že je připraveno přijmout data.

    Proces handshakingu:

    - Vysílání dat: Vysílací zařízení přenáší data sériově po UART lince.
    - RTS aktivace: Když je vysílací zařízení připraveno na přenos dalších dat, aktivuje signál RTS.
    - Čekání na CTS: Přijímací zařízení monitoruje signál RTS. Jakmile ho detekuje, pošle signál CTS zpět k vysílacímu zařízení.
    - Pokračování v přenosu: Po obdržení signálu CTS může vysílací zařízení pokračovat v přenosu dat.

- **Použití**
  - UARTy se používají v mnoha různých aplikacích, včetně sériových portů na počítačích, GPS modulů, rádiových modulů a dalších zařízení, která potřebují komunikovat přes sériovou linku.
- **Nevýhody**
  - UART má také své omezení. Například, vzhledem k tomu, že neexistuje žádný společný hodinový signál, odesílatel a příjemce musí být přesně synchronizováni, aby bylo možné správně interpretovat data. To může být problematické při vysokých rychlostech přenosu nebo při dlouhých vzdálenostech, kde může dojít k významnému zpoždění signálu.